SPEAK

Language-Learning Platform for Primary-School Children

Software Design Document

Prepared on: 11 July 2026

Prepared By

Janaki Badu

Full-Stack Web Developer

Client: SPEAK Language-Learning Programme
Status: For Review and Approval

Companion document to the SPEAK Software Requirements Specification (07 July 2026)

1. Introduction

SPEAK is a web-based platform that teaches German vocabulary and grammar to primary-school children through short, guided, audio-first activities. This document describes how the platform specified in the SPEAK Software Requirements Specification (SRS, 07 July 2026) will be designed and built: its architecture, its domain and data model, its frontend and backend structure, its security design, the quality gates that govern delivery, and the deployment approach.

The learning material was originally authored as psychology-experiment activities (OpenSesame and OSWeb) and delivered through the JATOS research platform. This project rebuilds that material as a purpose-built web application. The interactive task engine and the management features are developed from scratch; the existing media assets and the task content held in CSV files are reused and imported into the new system.

1.1 Purpose of this Document

This design document translates the requirements into an implementable technical design. It is intended to be read alongside the SRS. It defines the components to be built, the interfaces between them, the database structure, the security controls, and the conditions under which work is considered complete and releasable.

1.2 Relationship to the Requirements Specification

The SRS is the authoritative statement of scope. Where this design document and the SRS appear to conflict, the SRS prevails. Every design decision below is traceable to one or more requirement identifiers (R, NFR, TR), and Appendix B provides the full traceability matrix. Two points of the SRS are load-bearing for this design and are highlighted here:

- Three mandatory roles. Administrator, Teacher, and Student are all mandatory in the first release (SRS section 1.3, R03, R08, R09). The Administrator is therefore designed as a first-class role with its own screens, endpoints, and content-import capability, and is not treated as an operator or developer function.
- Data protection is a requirement, not an enhancement. The platform processes data about children in Germany. GDPR compliance, EU-resident storage and processing, data minimisation, and server-side access control (NFR01 to NFR15, TR10) constrain the architecture and the deployment target, and are designed in from the start.

1.3 Design Principles

The following principles are applied throughout and explain most of the decisions in this document.

Principle	Statement	Requirements
Data-driven task engine	The task player is a generic engine that renders content described by data. A new week of content is an import, not a code change.	NFR21, R42, R20
Server is authoritative	Frontend guards are a usability feature. Every access-control rule, scoring rule, and adaptive rule is enforced and re-verified on the server.	R14 to R16, NFR15
Content and learner data are separate	Learning content is imported, versioned, and replaceable. Learner responses are immutable and never overwritten by a content update.	NFR22
Minimise personal data	A child is identified by a display name or pseudonym and an access code. Nothing else about the child is stored.	NFR02

Principle	Statement	Requirements
Audio-first, reading-free child UI	The child interface is operable without fluent reading: large targets, spoken instructions, minimal text.	R39, NFR16
EU by construction	All structured data, media, backups, and logs remain within the European Union.	NFR01, TR10
Testable by construction	Access control, the adaptive rule, and the content importer are pure, independently testable units.	TR09

2. System Overview

SPEAK follows a classic three-tier architecture: a React single-page application (presentation), a Node.js and Express REST API (application logic), and a SQL relational database with an associated media store (persistence). A content importer converts the supplied CSV files and media assets into the system's own content format, so that the running application never depends on the original research files.

2.1 Technology Stack

Layer	Technology	Notes
Frontend	React (JavaScript)	Function components and hooks; reusable task-player components (TR01)
Build tooling	Vite	Fast dev server, bundling, environment-specific builds
Routing	React Router	Client-side routing with protected route components (TR01, TR02, R38)
State and logic	React Context and custom hooks	AuthContext for session and role; per-feature hooks
HTTP client	Axios	Single API layer that attaches the session token and normalises errors
Audio and media	HTML5 Audio with a preload service (Howler.js optional)	Preloading and sequenced playback of instruction and stimulus audio (R29, NFR19)
Backend	Node.js and Express	REST API; guards, task rules, progress, reporting (TR03)
Database	MySQL (SQL relational)	Accounts, content, assignments, attempts, responses, audit log (TR04)
Authentication	JWT for administrators and teachers; code-based session for students	bcrypt password hashing; signed tokens with inactivity expiry (TR05, NFR08, NFR09)
File handling	Multer	Content-import uploads and optional spoken-answer recordings (TR08)
Charts	Chart.js (react-chartjs-2)	Teacher and administrator progress dashboards (R36, R40)
Content import	csv-parse plus an import service	Converts the supplied CSV and media into content rows (TR07, R09)
Testing	Vitest / Jest, Supertest, Playwright	Unit, API integration, and end-to-end tests (TR09)
Packaging	Docker and Docker Compose	Identical images across environments (TR10)

2.2 Architecture Overview

React components never call the backend directly. They use hooks and a central API layer, which is the only code that talks to the REST API. On the server, no route handler is reached before the middleware chain has verified the session token, the role, and the ownership of the requested data.

Component	Responsibility
React SPA (frontend)	User interface and routing; the interactive task player; media playback; teacher and administrator dashboards; route and role guards (R11 to R13)

Component	Responsibility
Node.js / Express API	Authentication and authorisation, content delivery, response capture, adaptive-flow decisions, scoring, progress and reporting, media authorisation (R14 to R16)
SQL database (MySQL)	Users, classes, student accounts, content (modules, tasks, trials, media registry), assignments, attempts, responses, audit log
Media store	Images and audio for tasks; optional encrypted recordings of spoken answers (NFR07, NFR13)
Content importer	Administrator-triggered job that validates the CSV and media, normalises them, and writes a new content version (R09, R42, TR07)

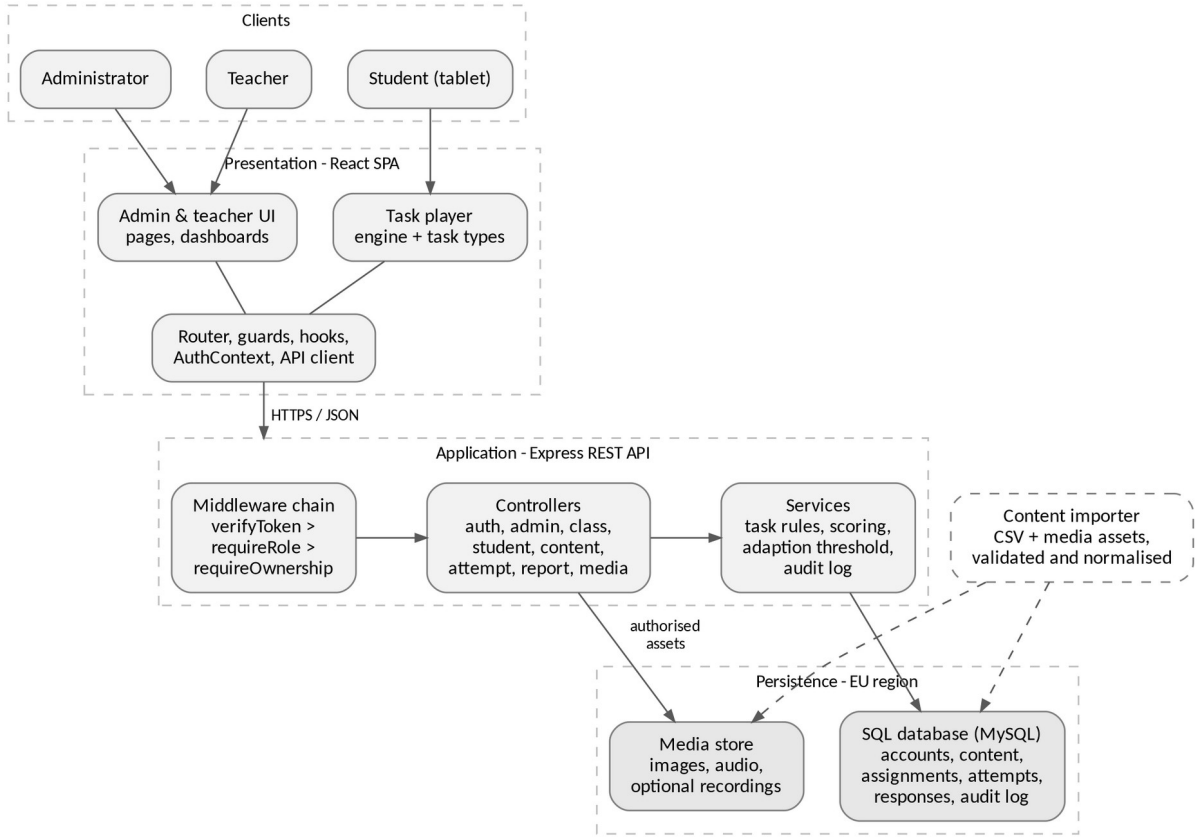


Figure 1: Architecture-level overview

2.3 Layer Rules

- A React component may not call axios directly; it calls a hook, which calls the API layer.
- An Express route handler may not read the request user from the token itself; it reads the user attached by the middleware chain.
- A service may not build SQL by string concatenation; all queries are parameterised (NFR12).
- A media asset is never served from a public static directory; it is served by a controller that checks authorisation first (NFR13, TR08).

3. Domain Model

The domain has three clearly separated groups: accounts (who uses the system), learning content (what is taught, imported and versioned), and learner data (what each child did). The domain model is framework-independent; it is realised on the backend as SQL tables and consumed by React as JSON.

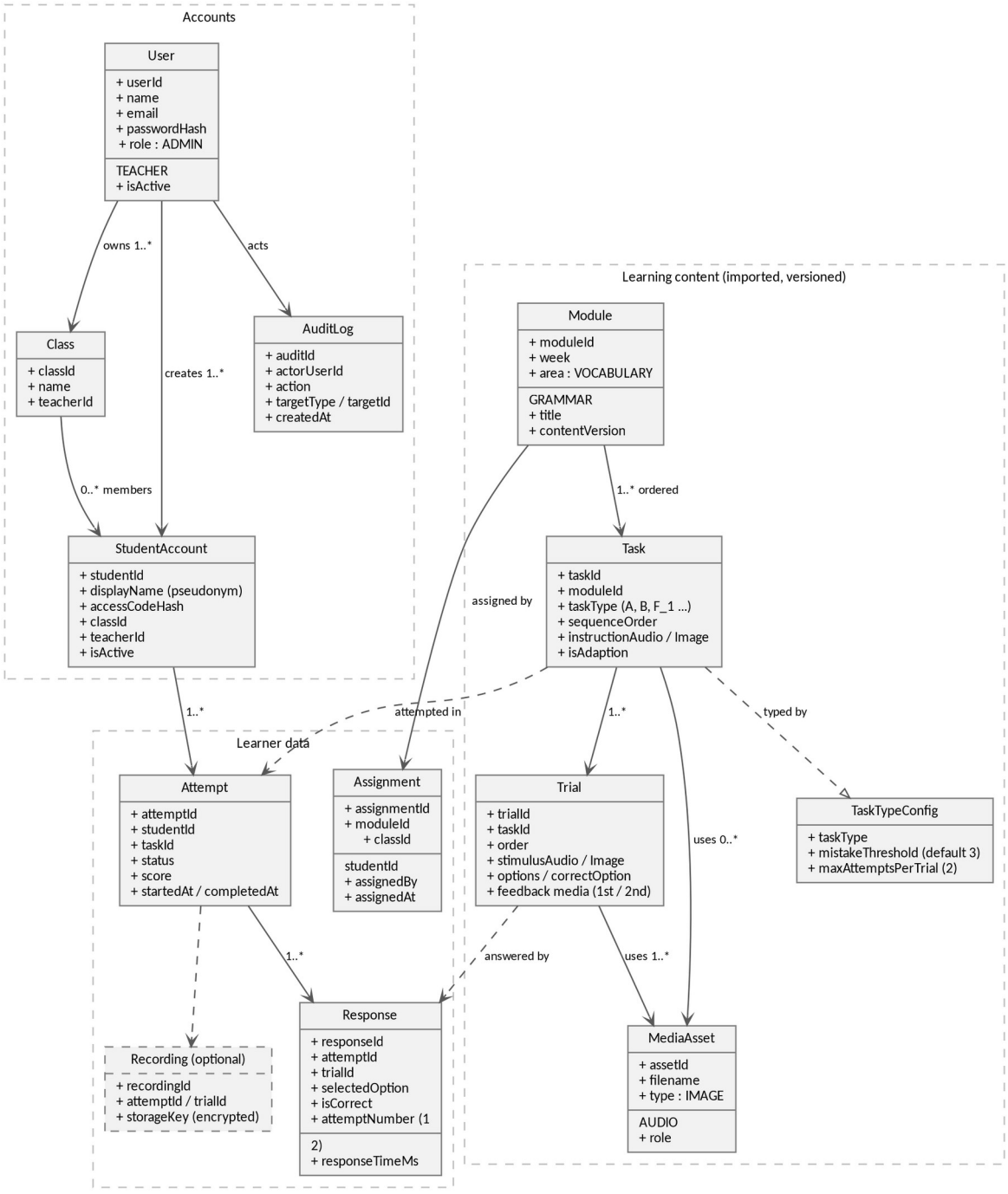


Figure 2: Domain-level class diagram

3.1 Content and Learner Data Separation

Learning content is imported and shared; attempts and responses belong to individual children. The two are never mixed in one table and never share a lifecycle (NFR22). Because content can be reimported or corrected, each module carries a content version, and every attempt records the content version it was

played against. A reimport therefore cannot silently alter or invalidate results that were already recorded (R33, R34).

3.2 The Data-Driven Task Engine

The Module – Task – Trial composition is the heart of the design. A module is one week within one area (R18). A task is an ordered block of one task type. A trial is a single question: a stimulus, a set of options, a correct option, and the feedback media for a correct answer, a first incorrect attempt, and a second incorrect attempt.

Because every task type in the supplied content reduces to this shape, the system does not implement eleven separate activities. It implements one engine and a small family of presentation components. The engine owns everything that is common to all task types:

- the trial sequence and the two-attempt rule (R31);
- immediate visual and audible feedback after each answer (R30);
- recording of every response (R33);
- mistake counting and the remediation threshold (R32);
- completion, scoring, and resume (R34, R35).

Each task type supplies only a presentation component that renders the trial data and reports the chosen option. Adding a new week of content means importing rows; adding a new task type means adding one presentation component (NFR21, R42).

Code	Task type	Presentation component
A	Storybook listening with word definitions (R21)	StorybookTask
B	Audio to image selection (R22)	AudioImageTask
F_1, F_2	Image to audio with a sound-alike distractor (R23)	AudioChoiceTask
T	Picture choice from a spoken target word (R24)	PictureChoiceTask
R	Yes / no reflection with image support (R25)	YesNoReflectionTask
G1, G2, V_3	Grammatical cases: article selection and gap-fill (R26)	GrammarArticleTask
U	Memory (suitcase) task (R27)	MemoryTask
AT, AV, AX	Guided teaching and reflection screens (R28)	TeachingReflectionTask
C	Remediation presented above the mistake threshold (R32)	Rendered by the engine using the type of the adaption task

4. Frontend Design (React)

The frontend is a React single-page application organised into feature folders rather than technical layers: src/features/admin, src/features/teacher, src/features/student, src/features/player, plus src/api, src/hooks, src/context, and src/components/common.

4.1 Pages

- App – root component; sets up the router and global providers.
- Layout and NavMenu – shared layout and navigation that shows only the items permitted for the authenticated role (R13).
- LoginPage – email and password authentication for teachers and administrators, with role-based redirect (R02, R03).
- StudentLoginPage – access-code entry only, large and child-friendly (R04).
- TeacherRegisterPage – teacher self-registration: name, email, password (R01).
- AdminDashboardPage – system overview: teachers, content versions, recent administrative actions.
- TeacherAccountsPage – create, activate, and deactivate teacher accounts (R08).
- ContentImportPage – upload and validate a CSV and media bundle, preview the result, and publish a content version (R09, R42).
- TeacherDashboardPage – classes, assignments, and class-level completion metrics (R36, R40).
- ClassManagementPage – create classes, add and edit students, display and regenerate access codes (R05, R06, R07).
- AssignmentPage – assign a module to a class or to an individual student (R19).
- ResultsPage – per-student and per-task results, filters, printable or exportable summary, module reset (R37, R41, R43, R45).
- StudentHomePage – the child's assigned modules as large, friendly tiles.
- ModulePlayerPage – runs a module as an ordered sequence of tasks and resumes an interrupted module (R18, R35).

4.2 The Task Player Component Family

- ModulePlayer – walks the ordered task list of a module, holds the resume point, and hands control to the TaskPlayer for each task.
- TaskPlayer – the generic engine: preloads media, runs the trial loop, applies the two-attempt rule, plays feedback, posts each response, and evaluates completion.
- Task-type components – StorybookTask, AudioImageTask, AudioChoiceTask, PictureChoiceTask, YesNoReflectionTask, GrammarArticleTask, MemoryTask, TeachingReflectionTask. Each renders trial data and emits the selected option; none of them contains flow, scoring, or persistence logic.
- Feedback – child-friendly correct and incorrect feedback, distinguishing the first from the second incorrect attempt (R30, R31).
- ProgressBar and RewardScreen – progress within a module, and an encouraging completion screen (R49, optional).

4.3 Hooks, Context, and the API Layer

- AuthContext and useAuth – login and logout for all three roles, token storage, current user and role, session invalidation on logout (R10).

- apiClient – the single axios wrapper: attaches the token, handles 401 and 403 uniformly, and never exposes server error internals (NFR14).
- useAdmin – teacher-account management and content import.
- useClasses, useStudents, useAssignments – teacher management operations.
- useContent – fetches the module, task, and trial configuration for the player (R20).
- useMedia – preloads and sequences audio and images for a task before it begins (R29, NFR19).
- useAttempt – starts an attempt, records each response, and completes and scores the attempt (R33, R34).
- useProgress and useReports – dashboards, filters, exports (R36, R41, R45, R48).
- useAudioRecording – optional capture and upload of spoken answers, enabled only where consent applies (R47, NFR07).

4.4 Routing and Access Control on the Client

Access control on the client prevents unauthorised navigation and hides what the user may not use. It is a usability layer only; it is never the security boundary (NFR15).

Wrapper	Behaviour	Requirement
ProtectedRoute	Redirects an unauthenticated user who requests a protected route to the appropriate login page (teacher / administrator login, or student code entry).	R11
RoleRoute	Permits the route only if the authenticated user holds the role that the route requires (ADMIN, TEACHER, STUDENT).	R12
NavMenu filter	Renders only the navigation items permitted for the current role.	R13
apiClient interceptor	On a 401 clears the session and returns to login; on a 403 shows an explicit authorisation error and no data.	R16

4.5 The Child Interface

The student interface is designed for a child who cannot yet read fluently (R39, NFR16). Instructions are spoken, not written. Interactive targets are large enough for a fingertip on a tablet, with generous spacing to prevent mis-taps. Text is used only as a label alongside an image or an audio button, never as the sole carrier of meaning. Feedback is both visual and audible. The layout is designed tablet-first and scales up to desktop (NFR17).

4.6 Media Preloading

A task does not begin until every audio and image asset it needs has been fetched and decoded (NFR19). The useMedia hook resolves a preload promise for the whole task; the TaskPlayer renders a short waiting state until it settles. A stutter in the middle of a listening task is treated as a defect, not as a performance issue.

5. Backend Design

The backend is a Node.js and Express REST API providing authentication, authorisation, content delivery, response capture, adaptive-flow decisions, reporting, and authorised media serving. Structured data lives in MySQL; media lives in an EU-resident object store with metadata in the database.

5.1 Key Concerns

- Authentication – JWT for administrators and teachers; a separate, scoped, short-lived session for student access codes (TR05).
- Authorisation – a fixed middleware chain that runs before every protected handler (TR06).
- Content delivery – endpoints that return module, task, and trial configuration as JSON for the task player (R20).
- Response capture – lightweight, high-frequency writes, one row per answer (R33).
- Adaptive flow – the mistake threshold is evaluated on the server; the client may predict the outcome for responsiveness but never decides it (R32).
- Progress and reporting – aggregation endpoints for the teacher and administrator dashboards (R36, R37, R40, R41).
- Content import – an administrator-triggered service that validates, normalises, and publishes a content version (R09, R42, TR07).
- Auditing – every account creation and deletion records the acting user and the time (NFR05).

5.2 The Middleware Chain

Server-side middleware verifies the token and the role of the requesting user before any protected route handler is executed (TR06). A third stage, ownership, is what prevents a legitimately authenticated user from reading another user's data (R15, NFR04). It is implemented once, as a reusable middleware, and unit-tested directly.

Stage	Question it answers	On failure	Requirement
verifyToken	Is the session token present, correctly signed, and unexpired?	401, no protected data returned	R14, NFR09
requireRole	Does this user hold the role that this route requires?	403, explicit authorisation error	R12, R16
requireOwnership	Does the requested student, class, assignment, or result actually belong to this teacher, or to this child?	403 (or 404 where existence itself is sensitive)	R15, NFR04
rateLimit	Have too many consecutive failed logins or access-code attempts come from this source?	429, temporary lockout	NFR11
errorHandler	Is anything internal about to leak?	Generic message; details logged server-side only	NFR14

5.3 Task Rules and the Adaptive Flow

The engine rules live in one service so that they can be tested in isolation and changed without touching controllers or components.

- Two attempts per trial. A first incorrect answer produces first-try feedback and the trial is retried; a second incorrect answer produces second-try feedback and the trial ends (R31).

- Mistake counting. The number of mistakes in a task is counted across its trials.
- Remediation. When the mistake count reaches or exceeds the configured threshold, the engine inserts the adaption task that follows the current task (R32).
- Configuration. The threshold defaults to three and is stored per task type in task_type_config, not in code (R32).
- Scoring. The score of a task is derived on the server from the recorded responses, so that it cannot be set by the client (R34).

5.4 Progress, Resume, and Results

Every response is written as it happens, with the trial, the selected option, correctness, the attempt number, and the timestamp (R33). An attempt row holds the status and the resulting score for each student and task (R34). Because responses are persisted individually rather than at the end of a task, an interrupted session loses nothing: the child resumes the module at the task at which it was interrupted, and all previously recorded responses remain (R35, NFR20).

5.5 The Content Importer

The importer is a first-class, tested part of the system, not a developer script. It is triggered by an administrator from the ContentImportPage and runs in four stages (R09, R42, TR07):

1. Parse. Read the supplied CSV files, which contain a metadata header row and trial rows, and the accompanying media assets.
2. Validate. Check that every referenced image and audio file exists, that every trial has a correct option, and that every task type is known. A missing asset fails the import; it never reaches a child.
3. Normalise. Map the varying CSV column names of the different task types onto the single trial model, and register every media asset.
4. Publish. Write the modules, tasks, trials, and media registry as a new content version and make it available for assignment.

The import produces a report: rows read, assets registered, warnings, and errors. Because the running application reads only the imported content, the original research files are never a runtime dependency (R20).

5.6 Media and File Serving

Media is never exposed as a public static directory. A media controller resolves an asset identifier, checks that the requesting session is authorised to see it (that is, that the asset belongs to a module the child is assigned to, or to a teacher’s or administrator’s scope), and then streams the file or issues a short-lived signed URL (NFR13, TR08). Optional recordings of spoken answers are stored encrypted at rest in a separate location, are readable only by the child’s teacher and the administrator, and are deleted with the student account (R47, NFR07, NFR03).

5.7 Representative REST Endpoints

Method	Endpoint	Purpose	Access
POST	/api/auth/register	Register a teacher (R01)	Public
POST	/api/auth/login	Teacher or administrator login, returns a JWT (R02, R03)	Public
POST	/api/auth/student-login	Student access-code login (R04)	Public
POST	/api/auth/logout	Invalidate the current session (R10)	Authenticated

Method	Endpoint	Purpose	Access
GET / POST	/api/admin/teachers	List and create teacher accounts (R08)	Admin
PATCH	/api/admin/teachers/:id	Activate or deactivate a teacher (R08)	Admin
POST	/api/admin/content/import	Upload and validate a content bundle (R09, R42)	Admin
POST	/api/admin/content/:version/ publish	Publish a validated content version (R09)	Admin
GET / POST	/api/classes	List and create classes (R07)	Teacher
GET / POST	/api/students	List and create student accounts (R05)	Teacher
PATCH	/api/students/:id	Edit or deactivate a student account (R05)	Teacher (owner)
DELETE	/api/students/:id	Delete a student and all recorded responses (NFR03)	Teacher (owner), Admin
POST	/api/students/:id/access-code	Regenerate the access code (R06)	Teacher (owner)
GET	/api/modules	List modules by week and area (R17)	Authenticated
GET	/api/modules/:id/tasks	Tasks and trials for a module (R18, R20)	Assigned student, Teacher
POST	/api/assignments	Assign a module to a class or a student (R19)	Teacher
POST	/api/attempts	Start or resume an attempt (R35)	Student
POST	/api/responses	Record one trial response (R33)	Student (owner)
PATCH	/api/attempts/:id/complete	Complete and score an attempt (R34)	Student (owner)
POST	/api/attempts/:id/reset	Reset a module for a student (R43)	Teacher (owner)
GET	/api/progress	Progress summary for a class or student (R36, R40)	Teacher, Admin
GET	/api/results	Detailed results, filterable and exportable (R37, R41, R45, R48)	Teacher (owner)
GET	/api/media/:assetId	Serve an authorised image or audio asset (NFR13)	Authorised session

5.8 Error Handling

A single error handler converts every thrown error into a safe response. Clients receive a stable error code and a human-readable message; stack traces, SQL fragments, and database identifiers are never returned (NFR14). Full detail is written to the server log, which contains no personal data and no access codes.

6. Data Design

6.1 Schema Overview

The schema separates accounts, content, assignment, and response capture. Content tables are written by the importer; learner tables are written by the application.

Table	Purpose	Group
users	Administrator and teacher accounts: name, email, password hash, role, active flag (R01, R08, NFR08)	Accounts
student_accounts	Pseudonymous student accounts: display name, hashed access code, class, owning teacher (R05, NFR02, NFR10)	Accounts
classes	Teacher-owned groups of students (R07)	Accounts
audit_log	Acting user, action, target, and timestamp for every account creation and deletion (NFR05)	Accounts
modules	One week within one area, with a content version (R17, R18)	Content
tasks	Ordered task blocks within a module: type, instruction media, adaption flag (R18)	Content
trials	Individual trials: stimulus, options, correct option, first- and second-try feedback media (R21 to R28)	Content
media_assets	Registry of every image and audio file, with type and role (R20, NFR13)	Content
task_type_config	Per task type: mistake threshold (default three) and attempts per trial (R31, R32)	Content
assignments	Module assigned to a class or to an individual student, with the assigning teacher and the date (R19)	Learner
attempts	A student's run at a task: status, score, timing, and the content version played (R34, R35)	Learner
responses	One row per answer: trial, selected option, correctness, attempt number, timestamp (R33)	Learner
recordings (optional)	Metadata and encrypted storage key for spoken answers, if enabled (R47, NFR07)	Learner

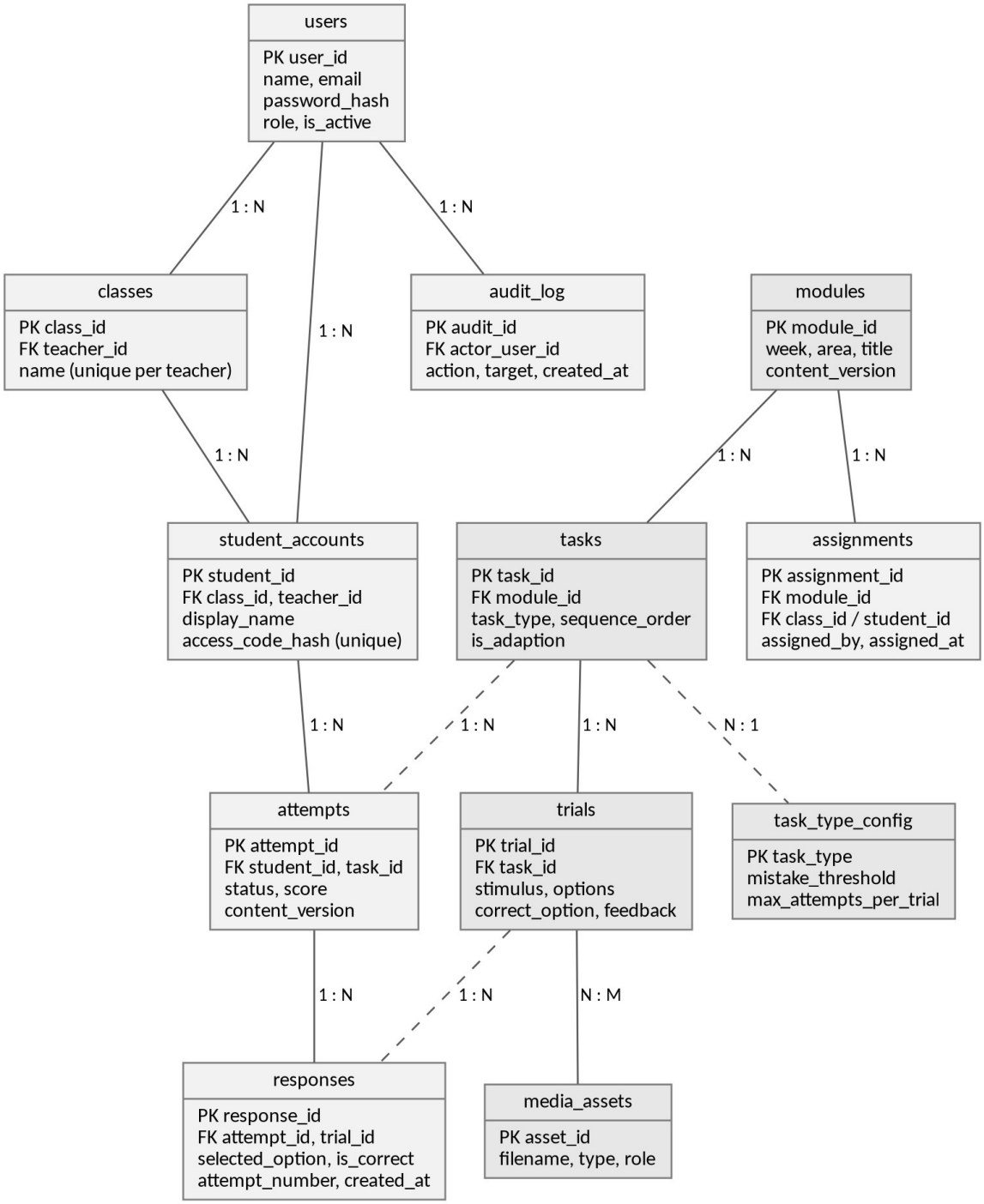


Figure 3: Entity-relationship overview

6.2 Constraints and Indexes

- Uniqueness. Teacher email is unique; a class name is unique per teacher; a student display name is unique per class; an access code is unique across the entire system (R46, NFR10).
- Access codes are stored hashed, not in clear text, and are shown to the teacher only at generation and regeneration (R06, NFR08).
- Foreign keys. student_accounts references classes and users; attempts references student_accounts and tasks; responses references attempts and trials.
- Cascading deletion. Deleting a student account deletes its attempts, responses, and recordings in one transaction (NFR03).

- Indexes. responses (attempt_id), attempts (student_id, task_id), assignments (class_id, student_id), trials (task_id, order) support the player and the dashboards without table scans.
- Queries are parameterised everywhere; no query is assembled by string concatenation (NFR12).

6.3 Retention and Deletion

A teacher or an administrator can permanently delete a student account together with every recorded response and any recording (NFR03). Deletion is a hard delete of learner data, not a flag, and is recorded in the audit log (NFR05). Learning content is never deleted by this operation.

7. Behavioural Design

7.1 Actors

- Administrator – logs in with email and password, manages teacher accounts, imports and updates learning content, and oversees system configuration.
- Teacher – registers and logs in, manages students, access codes, and classes, assigns modules, and reviews progress.
- Student (child) – logs in with an access code and works through assigned modules by listening, tapping, and answering.

7.2 Scenario A: Administrator Creates a Teacher Account

1. The administrator logs in and opens the teacher accounts page.
2. The administrator submits a name and an email address (POST /api/admin/teachers).
3. The middleware chain verifies the token and the ADMIN role before the handler runs.
4. The backend creates the account, writes an audit-log row with the acting user and the time, and returns the new teacher (R08, NFR05).
5. The user interface shows a confirmation message (R44).

7.3 Scenario B: Teacher Creates a Student and an Access Code

1. The teacher opens the class management page and adds a student with a display name and a class.
2. The backend validates uniqueness, generates a random access code of sufficient length, stores its hash, and returns the code once for the teacher to hand to the child (R05, R06, R46, NFR10).
3. The teacher can regenerate the code at any time, which invalidates the previous one.

7.4 Scenario C: A Child Completes a Task with Remediation

This is the central interaction of the platform and is shown in full below.

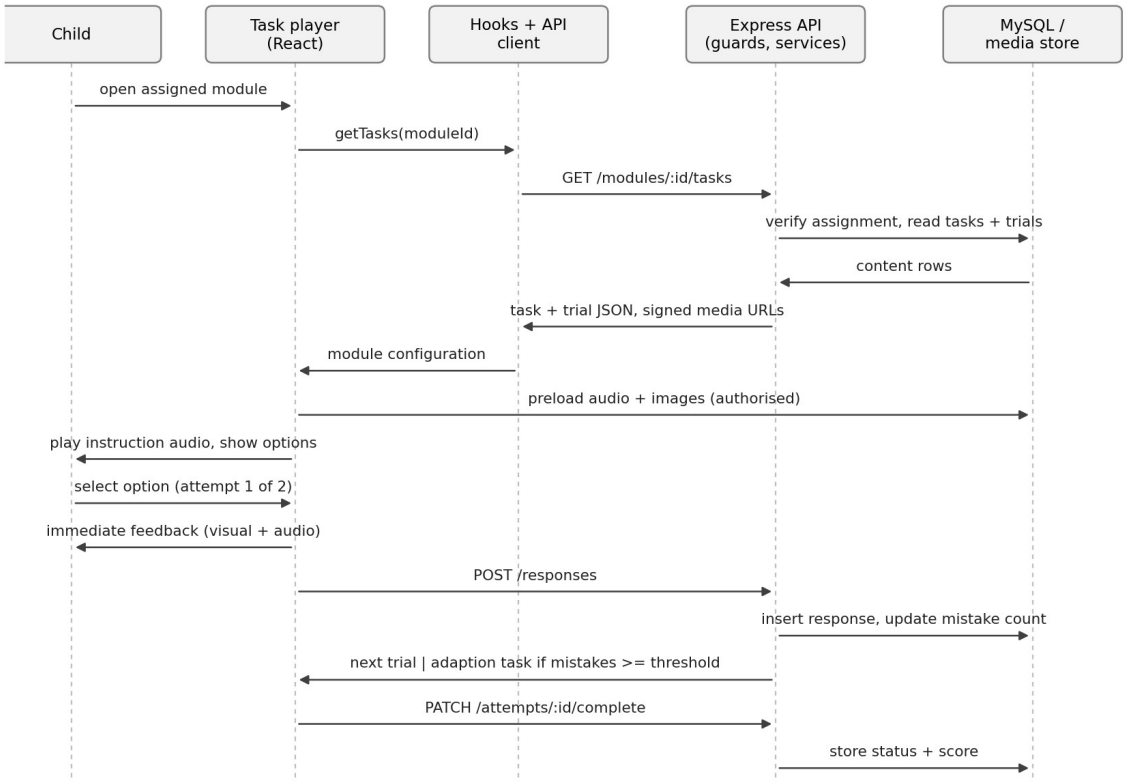


Figure 4: Interaction sequence – a child works through a task

The child logs in with the access code, opens an assigned module, and the player loads its tasks and trials. Media is preloaded before the first trial. For each trial the instruction and stimulus audio play, the options are shown, the child selects, and immediate feedback follows. Every answer is posted to the server as it happens. If the mistake count for the task reaches the configured threshold, the server directs the player to the remediation task before the module continues (R30 to R33). On completion the attempt is finalised and scored (R34).

7.5 Scenario D: Teacher Reviews Progress

1. The teacher opens the results page for a class or an individual student.
2. The reporting hook requests aggregated results (GET /api/progress, GET /api/results).
3. The ownership middleware confirms that the requested class and students belong to this teacher (R15).
4. The backend aggregates attempts and responses and returns the summary.
5. Results and charts are displayed and can be filtered by week, area, class, and student, and printed or exported (R36, R37, R41, R45).

7.6 Scenario E: Administrator Imports a New Week

1. The administrator uploads a CSV and media bundle on the content import page.
2. The importer parses, validates, and normalises it, and reports rows read, assets registered, warnings, and errors.
3. If validation passes, the administrator publishes the content version, which becomes available for assignment.
4. No application code is changed and no deployment is required (R42, NFR21).

8. Security and Data Protection Design

The platform processes data about children. The controls below are part of the design, not a hardening exercise applied afterwards.

8.1 Data Protection

Control	Design	Requirement
EU processing	Application servers, database, media store, backups, and logs are all hosted within the European Union.	NFR01, TR10
Data minimisation	A student record holds a display name or pseudonym and a hashed access code. No email address, no date of birth, no free-text notes about the child.	NFR02
Right to erasure	A single delete operation removes the student account, every response, and any recording, in one transaction.	NFR03
Need-to-know access	A child's data is reachable only by that child, the teacher who created the account, and the administrator; enforced by the ownership middleware.	NFR04, R15
Accountability	Every account creation and deletion writes an audit-log row with the acting user and the time.	NFR05
Encryption in transit	All traffic is served over HTTPS with TLS; HTTP is redirected and HSTS is enabled.	NFR06
Recordings	If spoken-answer recording is enabled, files are encrypted at rest, restricted to the child's teacher and the administrator, and deleted with the account.	NFR07, R47

8.2 System Security

Control	Design	Requirement
Password storage	bcrypt salted hashes; passwords are never stored or transmitted in clear text.	NFR08
Session tokens	Signed JWTs with an inactivity expiry for administrators and teachers; a separate scoped session for students.	NFR09, TR05
Access codes	Randomly generated, long enough to make guessing impractical, unique system-wide, and stored hashed.	NFR10
Brute-force protection	Consecutive failed logins and access-code attempts from the same source are rate-limited and then locked out temporarily.	NFR11
Injection prevention	All client input is validated and sanitised; all database access uses parameterised queries.	NFR12, R46
Protected assets	Media and uploaded files are served only to authorised sessions, never from a public directory.	NFR13, TR08
Safe errors	No stack traces, no SQL, no internal identifiers in any client response.	NFR14
Server enforcement	Every rule enforced by a frontend guard is independently enforced on the server.	NFR15, R14 to R16

9. Quality Assurance and Quality Gates

Quality is enforced by gates rather than by intention. Work does not advance to the next stage until the gate for the current stage passes.

9.1 Test Strategy

Level	Tool	Scope
Unit	Vitest / Jest	Task rules (two attempts, mistake counting, threshold, scoring), access-code generation, importer normalisation, guard middleware, React components in isolation
API integration	Supertest against a real MySQL container	Every protected endpoint exercised three ways: no token, wrong role, wrong owner; plus the happy path
Content import	Importer test suite against the client's real CSV and media	Every referenced image and audio file resolves; every trial has a correct option; unknown task types fail the build
End to end	Playwright	Teacher creates a student, assigns a module, the child logs in with the code, completes a trial, and the response appears on the dashboard
Manual	Checklist on a tablet and on the mainstream browsers	Child usability, audio behaviour, large targets, resume after interruption (NFR16 to NFR18, NFR20)

9.2 The Quality Gates

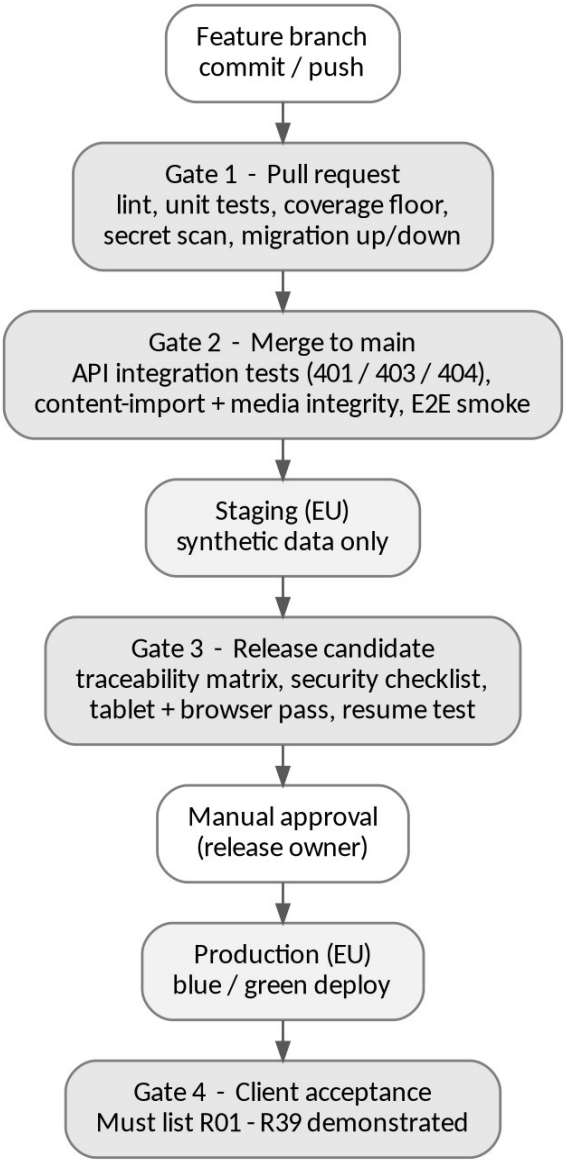


Figure 5: Delivery pipeline and quality gates

Gate 1 – Pull request (automated, blocking)

- ESLint and Prettier clean, with no warnings.
- Unit tests green. Coverage floor: eighty per cent overall, and one hundred per cent on authentication, the guard middleware, the ownership check, and the adaption and scoring rules (TR09).
- Dependency audit clean at high severity and above.
- No secrets in the diff.
- Database migrations run forward and backward on a clean database.
- The pull request cites the requirement identifiers it implements. A change that cites none is either scope creep or a missing requirement.

Gate 2 – Merge to main (automated, blocking)

- API integration tests pass, including the unauthenticated, wrong-role, and wrong-owner cases for every protected endpoint (R14 to R16, NFR15).

- The content importer runs against the supplied Weeks 6, 10, and 12 and every media reference resolves. A broken reference fails the build rather than reaching a child.
- The end-to-end smoke test passes.
- The application image builds and starts from a clean container.

Gate 3 – Release candidate (automated and manual)

- The requirements traceability matrix is complete: every Must requirement maps to a passing test or a signed-off manual check.
- Security checklist: HTTPS enforced, rate limits active, no internal error detail in responses, audit rows written, media not publicly reachable (NFR06, NFR11, NFR13, NFR14, NFR05).
- Accessibility and child-usability pass on a real tablet and on the current mainstream browsers (NFR16 to NFR18).
- Interruption test: a session is killed mid-module, resumed, and no recorded response is lost (R35, NFR20).
- A restore from backup has been rehearsed against the staging database.

Gate 4 – Client acceptance

- The Must requirements R01 to R39 are demonstrated to the client one by one against the matrix in Appendix B.
- Should and Can requirements are demonstrated where implemented but are not acceptance conditions unless promoted in writing.
- Acceptance is recorded against the SRS identifiers, not against screens.

9.3 Definition of Done

A unit of work is done when: the code is merged; unit and integration tests cover it; any migration is written and reversible; the requirement identifier is recorded; and the traceability matrix is updated. Nothing is done because it works on a developer machine.

10. Deployment and Operations

The binding constraint on deployment is NFR01 and TR10: everything, including backups and logs, remains within the European Union.

10.1 Environments

Environment	Purpose	Data
Development	Local Docker Compose stack: API, database, media store emulation	Synthetic seed data only
Staging	An identical EU-hosted copy of production, used for Gate 2 and Gate 3	Synthetic data only; never real children's data
Production	The live platform used by schools	Real learner data, EU-resident, backed up nightly

10.2 Target Infrastructure

The recommended target is a single EU virtual server running the application containers, with a managed MySQL instance and an EU object store for media. This is proportionate to the expected load (a small number of classes working through short tasks) and keeps data residency unambiguous.

Option	Region	Assessment
Hetzner Cloud (recommended)	Nuremberg or Falkenstein, Germany	German data residency, a data processing agreement, managed database and object storage, low cost. The simplest defensible answer for a German school context.
Scaleway	Paris, France	Equivalent EU residency and managed services; a reasonable alternative.
Render or Fly.io	Frankfurt or Amsterdam	Less operational work, higher cost, and the data processing agreement must be read carefully before it is relied on.
AWS eu-central-1	Frankfurt	Defensible and mature, but heavier and more expensive than this system needs.

10.3 Deployment Topology

- web – nginx (or Caddy) serving the built React bundle and reverse-proxying the API; TLS terminated here with Let's Encrypt, HTTP redirected, HSTS enabled (NFR06).
- api – the Node.js and Express container, stateless, horizontally scalable.
- database – a managed MySQL instance in the same EU region, not a container, so that backups, patching, and failover are handled by the provider (TR04, TR10).
- media – a private EU object-storage bucket; no public read access; assets reached only through the media controller or a short-lived signed URL (NFR13, TR08).

10.4 Continuous Integration and Delivery

GitHub Actions builds the image, runs Gate 1 and Gate 2, and deploys automatically to staging on a merge to main. Deployment to production requires a manual approval by the release owner. Automatic deployment to a system that holds children's data is deliberately not the default.

- Migrations run as a pre-deploy step; production migrations are forward-only, with a tested rollback path.
- The same image that passed staging is the image that is promoted; nothing is rebuilt for production.
- A release is tagged and mapped to the requirements it satisfies.

10.5 Configuration and Secrets

All configuration is supplied through environment variables: database credentials, the JWT signing key, the media bucket credentials, the token lifetime, and the rate-limit thresholds. Secrets are held in the platform secret store, never in the repository, and the signing keys differ per environment.

10.6 Backup and Recovery

- Nightly encrypted snapshot of the database, retained for thirty days, stored in the EU.
- Media bucket versioning enabled.
- A restore is rehearsed against staging before go-live and at least once per term. An untested backup is not a backup.

10.7 Monitoring and Logging

- Uptime and health checks on the API and the database.
- Structured application logs with no personal data, no access codes, and no tokens.
- Error tracking with the EU-hosted option and personal-data scrubbing enabled.
- The audit log required by NFR05 is stored in the database, not in the application log, so that it is queryable and retained.

11. Screen Inventory

The following screens make up the first release. High-fidelity mock-ups will be produced separately and attached before implementation; this inventory defines the purpose and key elements of each screen for that work.

Screen	Role	Key elements
Home / landing	Public	Introduction, teacher and administrator login, child “enter your code” entry
Teacher registration	Public	Name, email, password, confirmation (R01)
Login	Public	Email and password; redirect by role (R02, R03)
Student login	Public	Large access-code entry, no reading required (R04)
Administrator dashboard	Admin	Teachers, content versions, recent administrative actions
Teacher accounts	Admin	Create, activate, deactivate teacher accounts (R08)
Content import	Admin	Upload CSV and media, validation report, publish version (R09, R42)
Teacher dashboard	Teacher	Classes, assignments, class-level completion metrics, charts (R36, R40)
Class and student management	Teacher	Class list, add and edit students, display and regenerate access codes (R05 to R07)
Assignment	Teacher	Choose a module (week and area) and a class or student (R19)
Results and progress	Teacher	Per-student and per-task results, filters, export, module reset (R37, R41, R43, R45)
Student home	Student	Assigned modules as large, friendly tiles
Module player	Student	Sequential tasks, progress indicator, resume point (R18, R35)
Task screens (per type)	Student	Audio playback, image options, immediate feedback (R21 to R31)
Reward / completion	Student	Encouraging completion screen (R49, optional)

11.1 Delivery Phases

The phases are ordered so that the riskiest assumption – that one generic engine can render every task type – is tested early rather than late.

Phase	Content	Requirements
1. Skeleton and access control	All three roles, both credential types, the middleware chain, deployed to staging	R01 to R16, NFR08 to NFR15
2. Content model and importer	Import Weeks 6, 10, and 12; prove that every media reference resolves	R09, R17, R20, R42, TR07
3. Task engine and two task types	Types A and B on the generic engine; validate the abstraction	R18, R21, R22, R29 to R31
4. Remaining task types	Types F_1, F_2, T, R, G1, G2, V_3, U, AT, AV, AX	R23 to R28
5. Adaption, resume, response capture	Threshold remediation, interrupted-session resume, full response logging	R32 to R35, NFR20

Phase	Content	Requirements
6. Dashboards and administration	Teacher dashboard, results, filters, administrator screens	R36 to R41, R43 to R46
7. Optional items	Recording, export, gamification, multi-language	R47 to R50

Appendix A: Task-Type Catalogue

Derived from the supplied CSV files. Each task is a metadata header row plus trial rows; teaching screens use reflection rows. Recurring trial fields include a correct option, one to three distractors, and first- and second-try feedback audio, plus optional remediation follow-ups. This regularity is why a single configurable trial model covers every type.

Area	Types present	Representative trial fields
Vocabulary	A, B, C, F_1, F_2, T, R	stimulus audio and image; correct option; audio and image distractors; feedback for a correct answer; feedback for the first and the second incorrect attempt; follow-up if not correct
Grammar	G1, G2, U, V_3, AT, AV, AX, R	pre-sentence; image or images; article options with audio; correct article; whole sentence; nominative, accusative, and dative articles with audio for the reflection screens

The Vocabulary and Grammar CSV sets between them reference approximately 776 images and 1,279 audio files. The importer registers every one of them and fails the import if any is missing.

Appendix B: Requirements Traceability Matrix

Every requirement in the SRS is mapped to the section of this document that designs it and to the test level that will verify it. This matrix is the basis for Gate 3 and Gate 4.

Requirement	Design section	Verified by
R01 to R10 (accounts and authentication)	4.1, 5.1, 5.7, 7.2, 7.3	Unit and API integration
R11 to R13 (frontend guards)	4.4	Unit (component) and end to end
R14 to R16 (server-side access control)	5.2	API integration: no token, wrong role, wrong owner
R17 to R20 (content and assignment)	3.2, 5.5, 5.7, 6.1	Importer suite and API integration
R21 to R28 (interactive task types)	3.2, 4.2	Unit (per component) and end to end
R29 to R31 (media, feedback, two attempts)	4.6, 5.3	Unit and end to end
R32 (remediation threshold)	5.3, 6.1 (task_type_config)	Unit (rules service)
R33 to R35 (responses, scores, resume)	5.4, 6.1	API integration and interruption test
R36 to R39 (dashboard, results, navigation, child UI)	4.1, 4.5, 5.7	End to end and manual tablet pass
R40 to R46 (Should)	4.1, 5.7, 6.2	API integration and manual
R47 to R50 (Can)	5.6, 12	Deferred; verified if promoted
NFR01 to NFR07 (data protection)	8.1, 10.2, 10.6	Manual review and deployment checklist
NFR08 to NFR15 (system security)	5.2, 5.8, 8.2	Unit, API integration, security checklist
NFR16 to NFR18 (usability)	4.5	Manual tablet and browser pass
NFR19, NFR20 (performance and reliability)	4.6, 5.4	Unit and interruption test
NFR21, NFR22 (maintainability)	3.1, 3.2, 5.5	Importer suite: a new week is added with no code change
TR01 to TR10 (technical and acceptance)	2.1, 2.2, 4.4, 5.2, 5.5, 5.6, 9, 10	Build, test, and deployment evidence

End of Design Document